

UNIVERSIDAD NACIONAL DE TRUJILLO

CLASIFICACIÓN AUTOMÁTICA DE ACTOS DE DIÁLOGO EN REUNIONES MEDIANTE REDES NEURONALES CLÁSICAS E HÍBRIDAS

Estudio experimental con el corpus público MRDA/SILICONE

Escuela de Ingeniería de Sistemas · VII Ciclo

Curso: Ingeniería del Software

Docente: Santos Fernández Juan Pedro

Autores:

Castañeda Flores, Carlos Daniel — cdcastanedafl@unitru.edu.pe

Guzmán Limo, Lizbeth Dorita Anabel — ldguzmanli@unitru.edu.pe

Trujillo, Perú
Julio de 2026

Contenido

Resumen / Abstract

1. Introducción

2. Fundamentos y trabajos relacionados

3. Materiales y métodos

4. Resultados

5. Discusión

6. Conclusiones

7. Limitaciones y trabajo futuro

Referencias

Anexos técnicos y evidencias

Resumen

La investigación evaluó cinco arquitecturas neuronales para clasificar automáticamente actos de diálogo en transcripciones de reuniones. Se utilizó el conjunto público ICSI Meeting Recorder Dialogue Act (MRDA), distribuido mediante el benchmark SILICONE, con 109 228 intervenciones, 73 reuniones y 52 hablantes. El proceso incluyó limpieza, análisis exploratorio, entrenamiento de tres modelos clásicos (MLP con TF-IDF, CNN-1D y LSTM) y dos modelos híbridos (CNN-BiLSTM y BiLSTM con atención), evaluación sobre la partición oficial de prueba, validación cruzada estratificada por grupos con cinco folds, ajuste de hiperparámetros y pruebas no paramétricas. El mejor desempeño en prueba correspondió a MLP-TFIDF, con accuracy de 0,6427, F1 macro de 0,6219 y AUC ROC macro de 0,8896. En validación cruzada, CNN-1D presentó el mayor F1 macro promedio (0,4982) y la menor variación entre los modelos recurrentes e híbridos comparados. La prueba de Friedman identificó diferencias globales significativas ($\chi^2=17,28$; $p=0,0017$), aunque las comparaciones pareadas de Wilcoxon con corrección de Holm no resultaron significativas, debido a la limitada potencia estadística de cinco folds. El mejor modelo se persistió en formato HDF5 (.h5) y se integró en una API FastAPI y un dashboard autenticado en Streamlit, con exportación de reportes a PDF, Word y Excel. Los resultados constituyen una línea base reproducible, no un resultado de estado del arte, y muestran la viabilidad de incorporar clasificación de intenciones en un sistema de gestión de reuniones.

Palabras clave: actos de diálogo, reuniones, redes neuronales, MRDA, Streamlit, FastAPI, clasificación de texto.

Abstract

This study evaluated five neural architectures for automatic dialogue-act classification in meeting transcripts. The public ICSI Meeting Recorder Dialogue Act (MRDA) dataset distributed through SILICONE was used, comprising 109,228 utterances, 73 meetings, and 52 speakers. The workflow included cleaning, exploratory data analysis, training of three classical models (TF-IDF MLP, 1D CNN, and LSTM) and two hybrid models (CNN-BiLSTM and attention-based BiLSTM), evaluation on the official test split, five-fold stratified group cross-validation, hyperparameter tuning, and non-parametric statistical testing. TF-IDF MLP achieved the best test performance (accuracy 0.6427, macro-F1 0.6219, macro ROC-AUC 0.8896). CNN-1D showed the highest mean macro-F1 in cross-validation (0.4982). Friedman's test detected overall differences ($\chi^2=17.28$; $p=0.0017$), whereas Holm-adjusted Wilcoxon pairwise comparisons were not significant. The selected model was saved as an HDF5 artifact and integrated into FastAPI and an authenticated Streamlit dashboard. The results provide a reproducible baseline rather than a state-of-the-art claim.

Keywords: dialogue acts, meetings, neural networks, MRDA, Streamlit, FastAPI, text classification.

1. Introducción

Las reuniones generan decisiones, preguntas, acuerdos, retroalimentación y compromisos que suelen quedar dispersos en transcripciones o notas. La clasificación automática de actos de diálogo permite asignar una función comunicativa a cada intervención y puede servir como etapa previa para detectar preguntas, declaraciones, señales de seguimiento y otras intenciones relevantes para la gestión de reuniones.

El corpus MRDA fue creado a partir de reuniones multipersona del International Computer Science Institute y fue descrito por Shriberg et al. (2004). Posteriormente, SILICONE reunió MRDA y otros conjuntos de lenguaje hablado en un benchmark común para tareas de etiquetado secuencial (Chapuis et al., 2020). Esta disponibilidad pública permite construir experimentos reproducibles sin fabricar registros ni depender de información privada de usuarios.

El sistema base asignado al equipo ya gestionaba usuarios, reuniones, participantes, tareas, resúmenes y métricas mediante Streamlit, Supabase y automatizaciones n8n. No obstante, el consumo de un modelo generativo externo no constituía por sí mismo un experimento científico entrenado con datos públicos. Por ello, esta investigación incorporó un módulo de aprendizaje supervisado evaluado con cinco arquitecturas, reportes comparativos y pruebas estadísticas.

1.1 Problema de investigación

¿Qué arquitectura neuronal, entre tres modelos clásicos y dos modelos híbridos, ofrece el mejor equilibrio entre F1 macro, capacidad discriminativa, tiempo de procesamiento y estabilidad para clasificar actos de diálogo en transcripciones de reuniones del corpus público MRDA?

1.2 Objetivo general

Comparar cinco arquitecturas neuronales para la clasificación de actos de diálogo en reuniones, utilizando el corpus público MRDA y una plataforma integrada con Streamlit y FastAPI.

1.3 Objetivos específicos

- Realizar la limpieza y el análisis exploratorio del corpus, incluyendo distribución de clases, longitud de textos y estadísticos descriptivos.
- Entrenar tres modelos clásicos: MLP-TFIDF, CNN-1D y LSTM.
- Entrenar dos modelos híbridos: CNN-BiLSTM y BiLSTM con mecanismo de atención.
- Comparar accuracy, precisión macro, recall macro, F1 macro, F1 ponderado, ROC-AUC, tiempos, tamaño y número de parámetros.
- Aplicar validación cruzada de cinco folds configurable, tuning y pruebas estadísticas no paramétricas.
- Persistir el mejor modelo y consumirlo desde FastAPI sin reentrenamiento.
- Presentar resultados e interpretaciones en Streamlit y en reportes PDF, Word y Excel.

1.4 Hipótesis

H1: existe al menos una diferencia estadísticamente significativa en el F1 macro obtenido por las cinco arquitecturas evaluadas mediante validación cruzada. H0: no existen diferencias significativas entre sus distribuciones de F1 macro.

2. Fundamentos y trabajos relacionados

2.1 Clasificación de actos de diálogo

Un acto de diálogo expresa la función pragmática de una intervención. En la variante de cinco clases de MRDA/SILICONE, las etiquetas utilizadas fueron: declaración (s), pregunta declarativa (d), retroalimentación breve o backchannel (b), continuación/seguimiento o follow-me (f) y pregunta (q). La tarea se formuló como clasificación multiclase de una sola etiqueta por intervención.

2.2 Arquitecturas evaluadas

MLP-TFIDF representa cada intervención con ponderaciones término-frecuencia/inversa de frecuencia documental y utiliza una red multicapa. CNN-1D aprende filtros sobre secuencias de tokens y captura patrones locales. LSTM modela dependencias secuenciales mediante memoria y compuertas. CNN-BiLSTM combina detección local con contexto bidireccional. BiLSTM con atención pondera estados ocultos para concentrar la decisión en fragmentos informativos. Estas familias se sustentan en trabajos clásicos de Kim (2014), Hochreiter y Schmidhuber (1997), Schuster y Paliwal (1997) y Bahdanau et al. (2015).

2.3 Evaluación estadística

Además de comparar promedios, se aplicó la prueba no paramétrica de Friedman sobre los resultados de los folds. Ante significancia global, se realizaron comparaciones pareadas de Wilcoxon y se controló el error familiar mediante la corrección de Holm, siguiendo recomendaciones habituales para comparación de múltiples clasificadores (Demšar, 2006).

3. Materiales y métodos

3.1 Diseño

Se desarrolló un estudio cuantitativo, aplicado, experimental y comparativo. La unidad de análisis fue una intervención textual de reunión con etiqueta de acto de diálogo. El flujo completo fue reproducible mediante scripts Python: descarga, verificación de hashes, EDA, entrenamiento, validación cruzada, tuning, pruebas estadísticas, generación de reportes e integración.

3.2 Dataset público

Se descargaron las particiones públicas de MRDA desde el repositorio SILICONE: entrenamiento (83,943 registros), desarrollo (9,815) y prueba (15,470). El total procesado fue 109,228 intervenciones. Se conservaron las particiones oficiales y se registraron las sumas SHA-256 para verificar integridad. La ficha del benchmark declara licencia CC BY-NC-SA 4.0; por ello, los datos se usan con atribución y fines académicos no comerciales.

Archivo	Registros	SHA-256 (inicio)
mrda_train.csv	83943	d7a963aac70eb80d...
mrda_dev.csv	9815	dc795c60b3825645...
mrda_test.csv	15470	8426417d316cb0aa...

Tabla 1. Archivos públicos descargados y verificados.

3.3 Limpieza y EDA

Se verificaron identificadores duplicados, textos vacíos y valores nulos. No se hallaron duplicados en Utterance_ID ni campos vacíos en las variables utilizadas. Se calcularon longitud en caracteres y palabras, distribución por clase y partición, frecuencia de palabras y un mapa de calor de clases. No se eliminaron signos interrogativos ni palabras funcionales, porque aportan señales para diferenciar preguntas y retroalimentaciones.

3.4 Preparación de datos

Para la ejecución reportada se tomó una muestra estratificada reproducible de 30 000 intervenciones del split oficial de entrenamiento, manteniendo la evaluación sobre los 15 470 registros completos del split oficial de prueba y usando el split de desarrollo para control. Esta decisión redujo el costo de cómputo disponible y se declara como limitación. Para modelos secuenciales se construyó un vocabulario máximo de 12 000 tokens, longitud de secuencia de 40 y embeddings de 48 dimensiones. Para MLP se usaron 2 500 atributos TF-IDF. Se aplicaron ponderaciones de clase en la pérdida para mitigar el desbalance.

3.5 Configuración de modelos

Modelo	Categoría	Estructura general
MLP-TFIDF	Clásico	TF-IDF 2 500 → capas densas → 5 clases
CNN-1D	Clásico	Embedding → convolución 1D → pooling global → 5 clases
LSTM	Clásico	Embedding → LSTM → capa densa → 5 clases
CNN-BiLSTM	Híbrido	Embedding → CNN-1D → BiLSTM → capa densa
BiLSTM-Atención	Híbrido	Embedding → BiLSTM → atención → capa densa

Tabla 2. Modelos clásicos e híbridos evaluados.

La configuración común usó semilla 42, lote de 256, dos épocas en la comparación principal y tasa de aprendizaje inicial de 0,001. El número reducido de épocas evita presentar un ajuste excesivo bajo recursos limitados; por ello, los resultados deben interpretarse como una línea base reproducible y no como el máximo desempeño posible.

3.6 Métricas y visualizaciones

La métrica de selección fue F1 macro, porque otorga el mismo peso a cada clase ante una distribución desbalanceada. También se reportaron accuracy, precisión macro, recall macro, F1 ponderado, ROC-AUC macro

one-vs-rest, matriz de confusión, curvas ROC por clase, pérdida, tiempo de entrenamiento, tiempo de inferencia, milisegundos por registro, tamaño de archivo y parámetros entrenables.

3.7 Validación cruzada, tuning y pruebas estadísticas

La validación cruzada fue configurable y se ejecutó con cinco folds mediante StratifiedGroupKFold, usando Dialogue_ID como grupo para reducir la filtración de intervenciones de una misma reunión entre entrenamiento y validación. Se empleó una muestra estratificada de 8 000 registros para esta fase. El tuning del modelo CNN-BiLSTM exploró cuatro combinaciones de dimensión de embedding, unidades ocultas, dropout y tasa de aprendizaje sobre 20 000 registros. Finalmente se aplicó Friedman al F1 macro de los cinco folds y Wilcoxon-Holm en comparaciones pareadas.

3.8 Arquitectura de software

El frontend Streamlit conserva la autenticación y el dashboard del sistema original. Después de validar credenciales, el usuario accede al módulo “Inteligencia artificial”, donde puede clasificar una intervención, revisar EDA, comparar modelos, observar validación y descargar reportes. FastAPI expone salud, estado del modelo, predicción individual y por lote, métricas y descarga de archivos. El servicio carga directamente best_model.h5 y el vectorizador TF-IDF; por tanto, no reentrena en cada solicitud.

Endpoint	Propósito
GET /health	Verifica disponibilidad del backend
GET /model/status	Informa artefacto H5 y metadatos
POST /predict	Clasifica una intervención
POST /predict/batch	Clasifica hasta 100 intervenciones
GET /metrics	Devuelve comparación de modelos
GET /reports	Lista reportes disponibles
GET /reports/{archivo}	Descarga PDF, Word o Excel

Tabla 3. Endpoints implementados en FastAPI.

4. Resultados

4.1 Resultados del EDA

Indicador	Resultado
Registros totales	109,228
Reuniones únicas	73
Hablantes únicos	52
Textos vacíos	0
Duplicados Utterance_ID	0
Palabras promedio	6.89
Palabras mediana	4

Tabla 4. Resumen del análisis exploratorio.

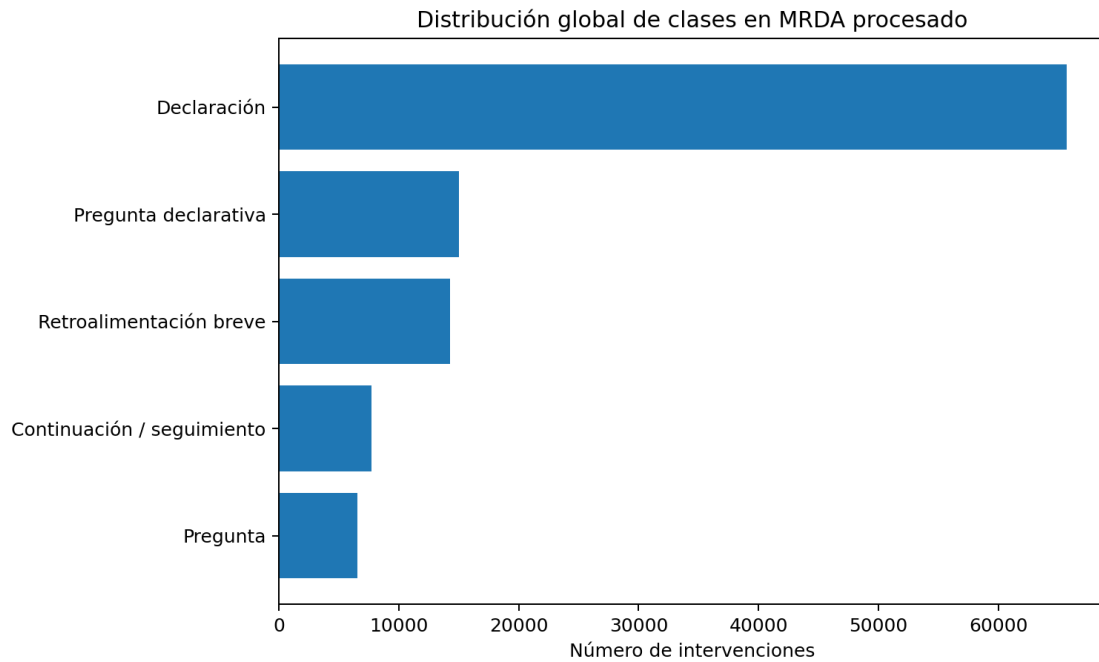


Figura 1. Distribución global de las cinco clases.

La clase “Declaración” concentra 65 691 registros (aproximadamente 60,1 % del total), mientras que “Pregunta” es la menos frecuente con 6 552. El desbalance justifica el uso de F1 macro y ponderación de clases; una accuracy alta por sí sola podría ocultar un desempeño deficiente en categorías minoritarias.

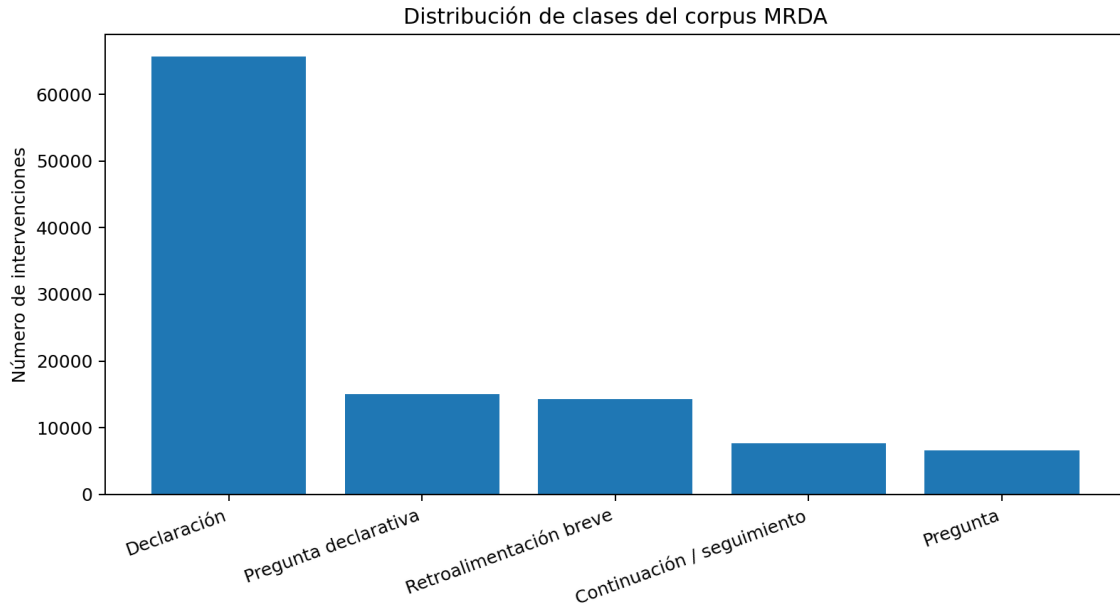


Figura 2. Distribución de clases en entrenamiento, desarrollo y prueba.

Las tres particiones mantienen una estructura semejante: predominan declaraciones, seguidas por preguntas declarativas y retroalimentaciones breves. Esta consistencia reduce el riesgo de que el conjunto de prueba presente una composición radicalmente distinta a la de entrenamiento.

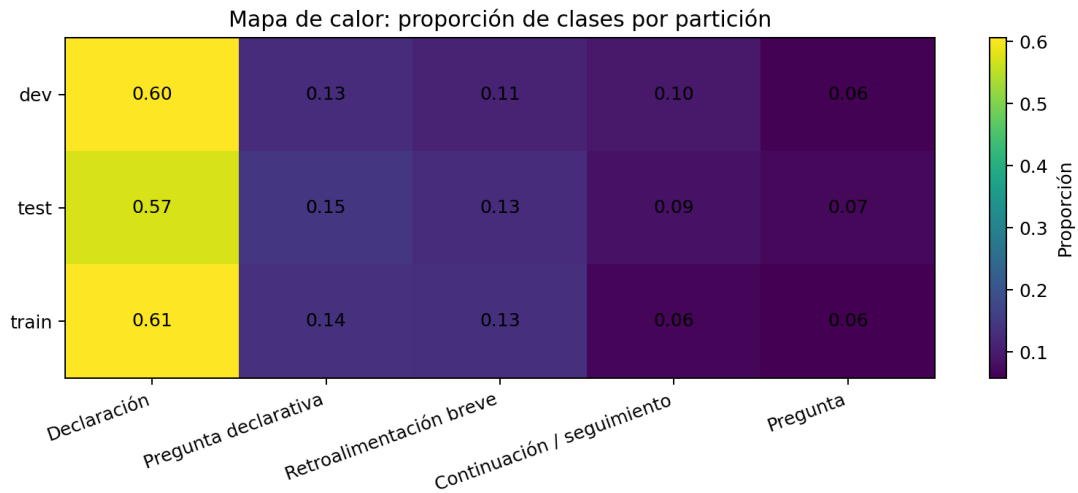


Figura 3. Mapa de calor de frecuencia por clase y partición.

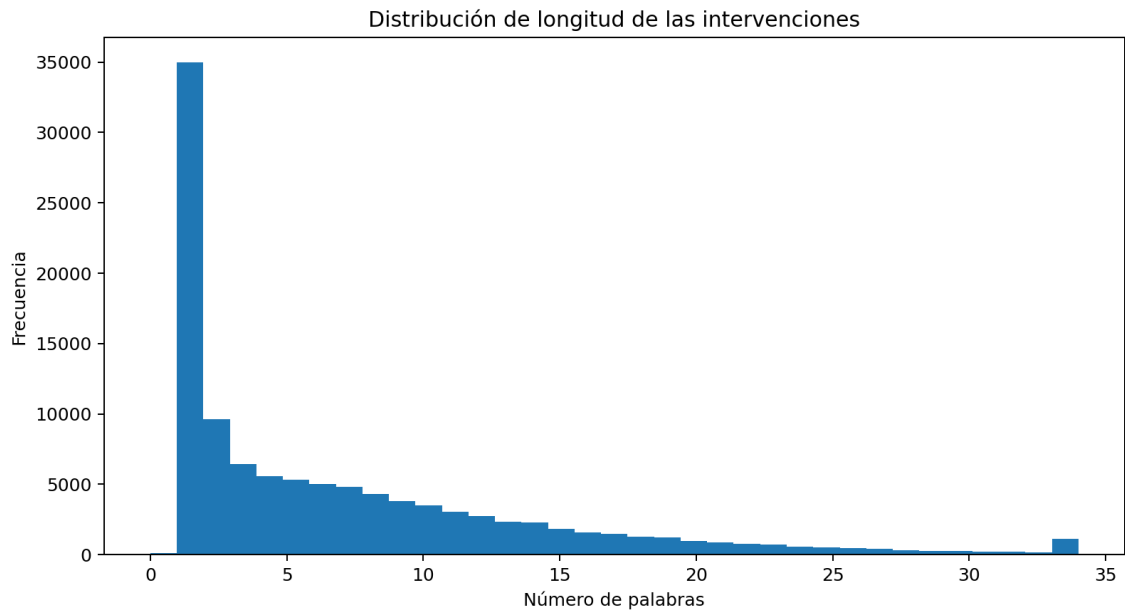


Figura 4. Distribución de longitud de intervenciones.

La mediana fue de cuatro palabras y el promedio de 6,89; se observó una cola larga de intervenciones extensas. La longitud máxima alcanzó 80 palabras. Una secuencia de 40 tokens cubre la mayor parte de ejemplos y limita el costo de los modelos recurrentes.

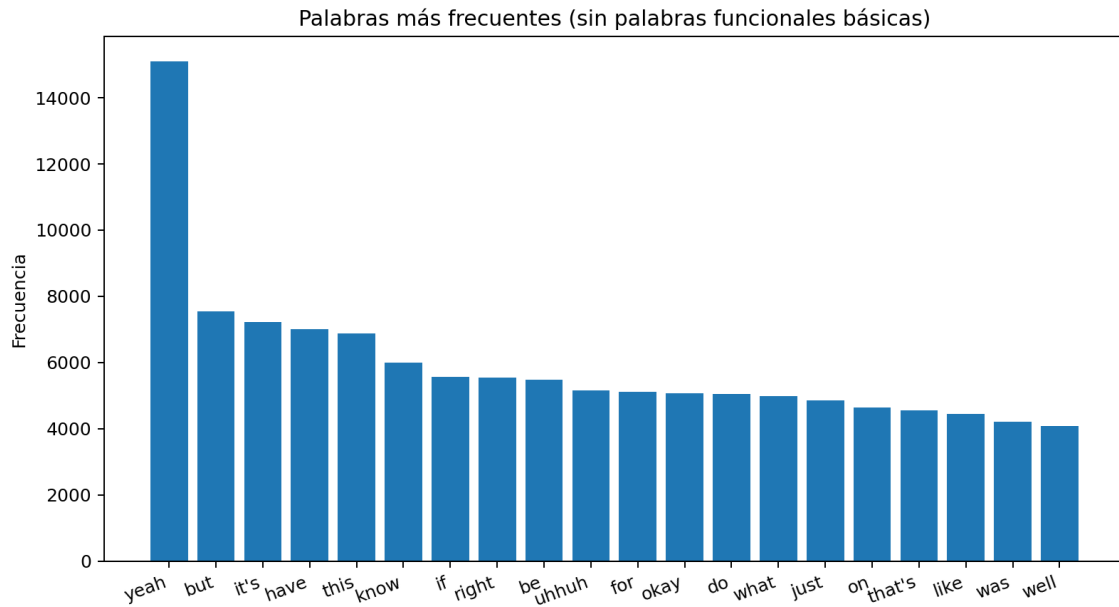


Figura 5. Palabras frecuentes en el corpus procesado.

4.2 Comparación de los cinco modelos

Modelo	Accuracy	F1 macro	ROC-AUC	Entren. (s)	Infer. (s)	MB
MLP-TFIDF	0.6427	0.6219	0.8896	1.59	0.077	1.26
CNN-1D	0.6488	0.5977	0.8737	2.70	0.098	1.38
BiLSTM-Atencion	0.5822	0.5801	0.8769	19.70	0.664	1.49
CNN-BiLSTM	0.5537	0.5626	0.8734	15.31	0.786	1.48
LSTM	0.5014	0.2204	0.6890	7.89	0.282	1.40

Tabla 5. Comparación principal sobre los 15 470 registros oficiales de prueba.

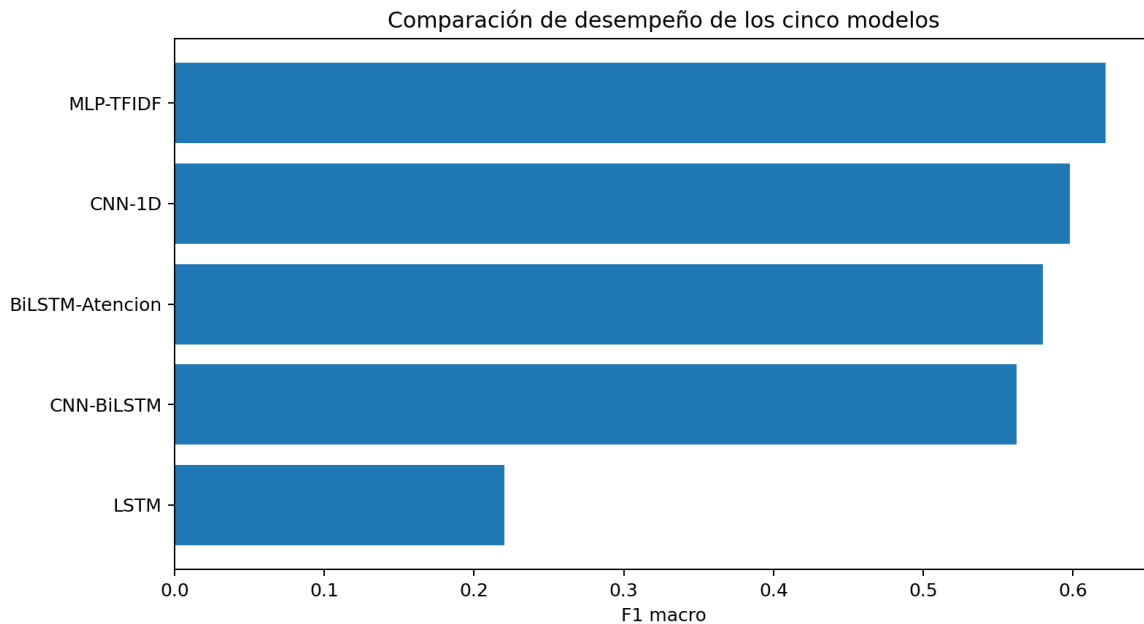


Figura 6. Comparación de F1 macro de los cinco modelos.

MLP-TFIDF obtuvo el mayor F1 macro (0,6219) y ROC-AUC macro (0,8896), además del menor tiempo de entrenamiento e inferencia. CNN-1D alcanzó la mayor accuracy (0,6488), pero su F1 macro fue inferior. Esto

muestra que la exactitud global y el equilibrio entre clases conducen a selecciones diferentes. El modelo LSTM presentó el menor F1 macro (0,2204), lo que evidencia que la arquitectura y el corto presupuesto de entrenamiento no fueron suficientes para representar adecuadamente las clases minoritarias.

4.3 Mejor modelo: matriz de confusión y curva ROC

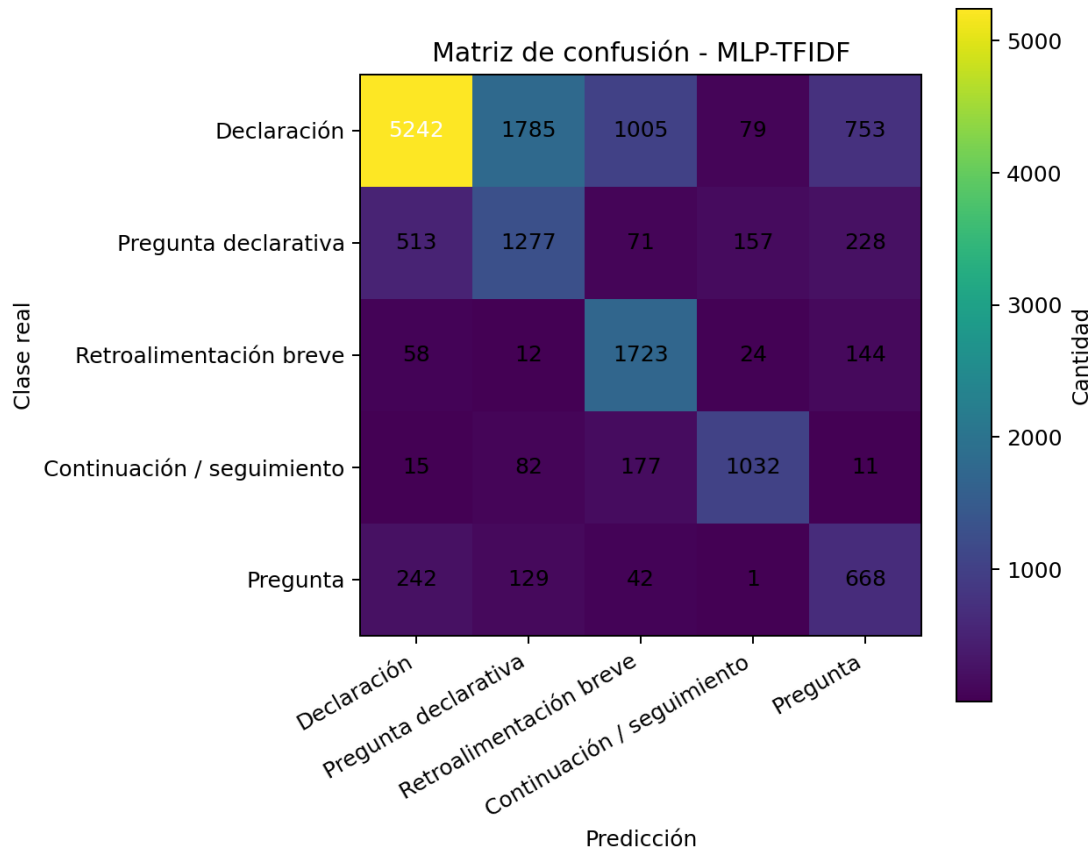


Figura 7. Matriz de confusión del modelo MLP-TFIDF.

La matriz revela una clasificación sólida de declaraciones, pero también confusiones entre clases cercanas y minoritarias. Las señales breves de retroalimentación y seguimiento son más difíciles de separar cuando se analiza una intervención sin contexto conversacional previo. Este patrón explica por qué el recall macro fue mayor que la precisión macro y por qué aún existe margen de mejora.

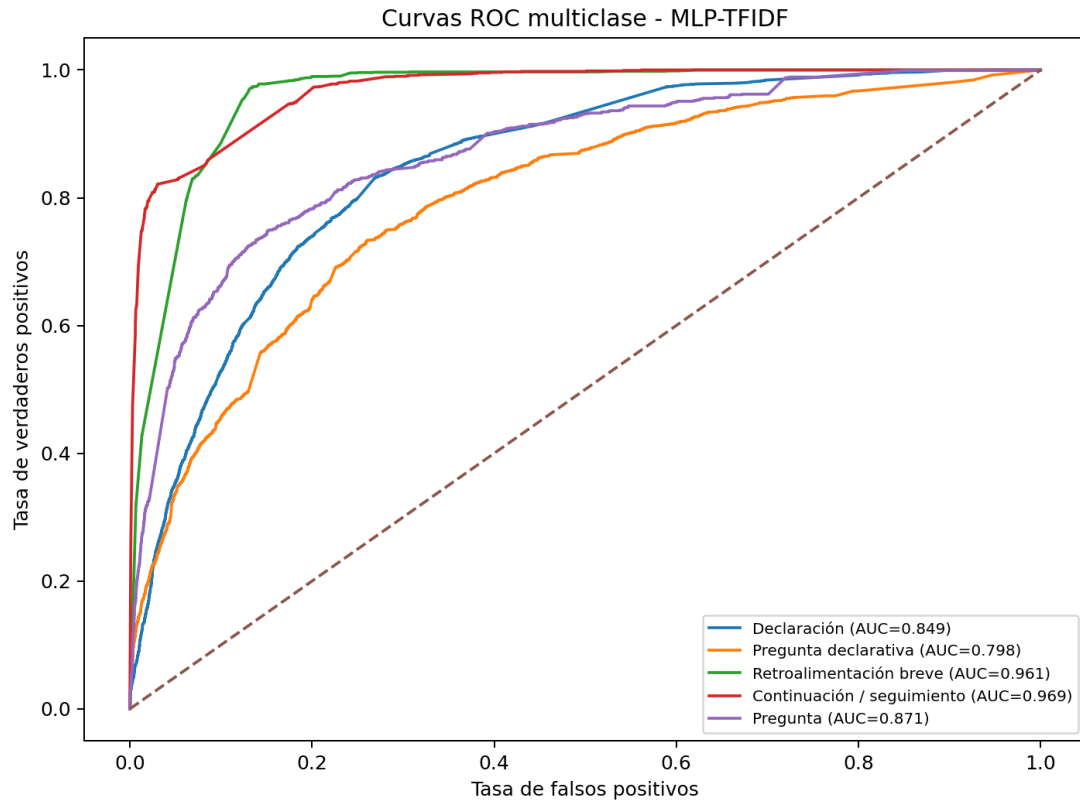


Figura 8. Curvas ROC one-vs-rest del modelo MLP-TFIDF.

El AUC macro de 0,8896 indica una capacidad discriminativa general favorable entre clases, aunque la curva ROC no sustituye el análisis de F1 ni la matriz de confusión en un problema desbalanceado.

4.4 Validación cruzada de cinco folds

Modelo	Acc. media	Acc. DE	F1 media	F1 DE	Tiempo medio (s)
CNN-1D	0.6195	0.0183	0.4982	0.0183	0.3931
MLP-TFIDF	0.6889	0.0270	0.4525	0.0328	0.2964
BiLSTM-Atencion	0.5058	0.1345	0.4079	0.0424	1.3642
CNN-BiLSTM	0.4420	0.1822	0.3820	0.0561	2.1419
LSTM	0.2250	0.2142	0.0713	0.0454	0.6509

Tabla 6. Resumen de validación cruzada agrupada y estratificada.

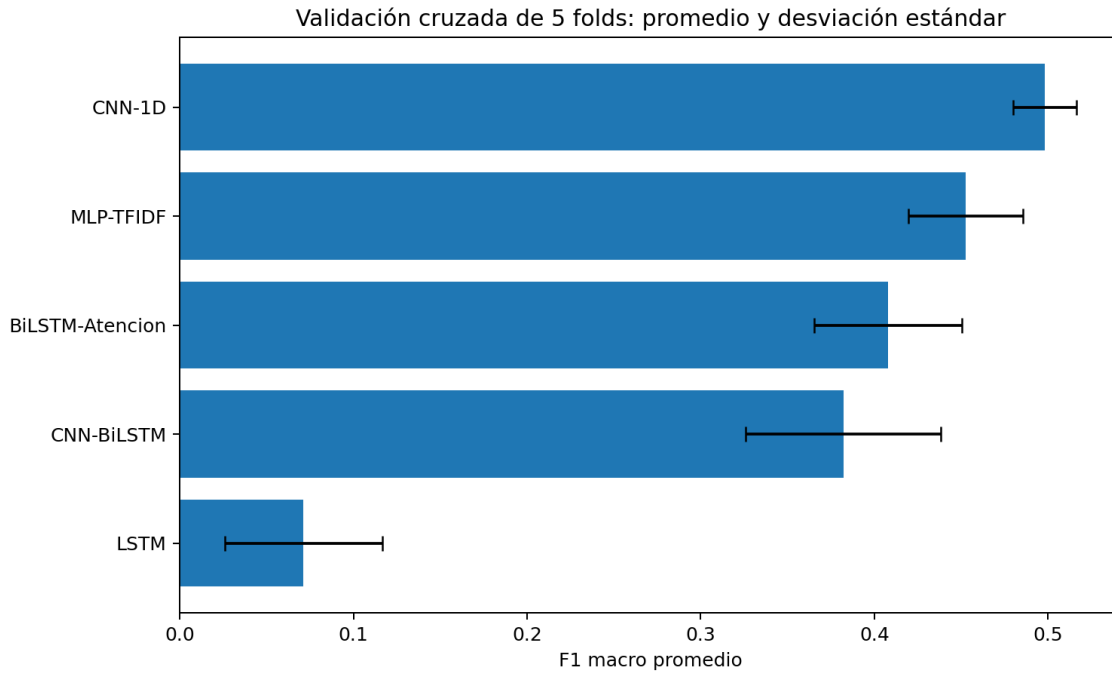


Figura 9. F1 macro por modelo en validación cruzada de cinco folds.

CNN-1D alcanzó el mayor F1 macro promedio (0,4982) y una desviación estándar de 0,0183, por lo que fue el modelo más estable en esta validación. MLP-TFIDF obtuvo la mayor accuracy promedio (0,6889), pero un F1 macro de 0,4526. Esta diferencia confirma que la clase mayoritaria influye en accuracy y refuerza la selección basada en F1 macro.

4.5 Ajuste de hiperparámetros

Ensayo	Emb.	Ocultas	Dropout	LR	Accuracy	F1 macro	Tiempo (s)
2	32	32	0.2000	0.0010	0.5494	0.5189	4.8021
4	32	32	0.3500	0.0010	0.5326	0.5130	4.7972
3	32	32	0.3500	0.0005	0.6376	0.4659	5.3255
1	32	32	0.2000	0.0005	0.2640	0.3288	8.0848

Tabla 7. Resultados del tuning de CNN-BiLSTM.

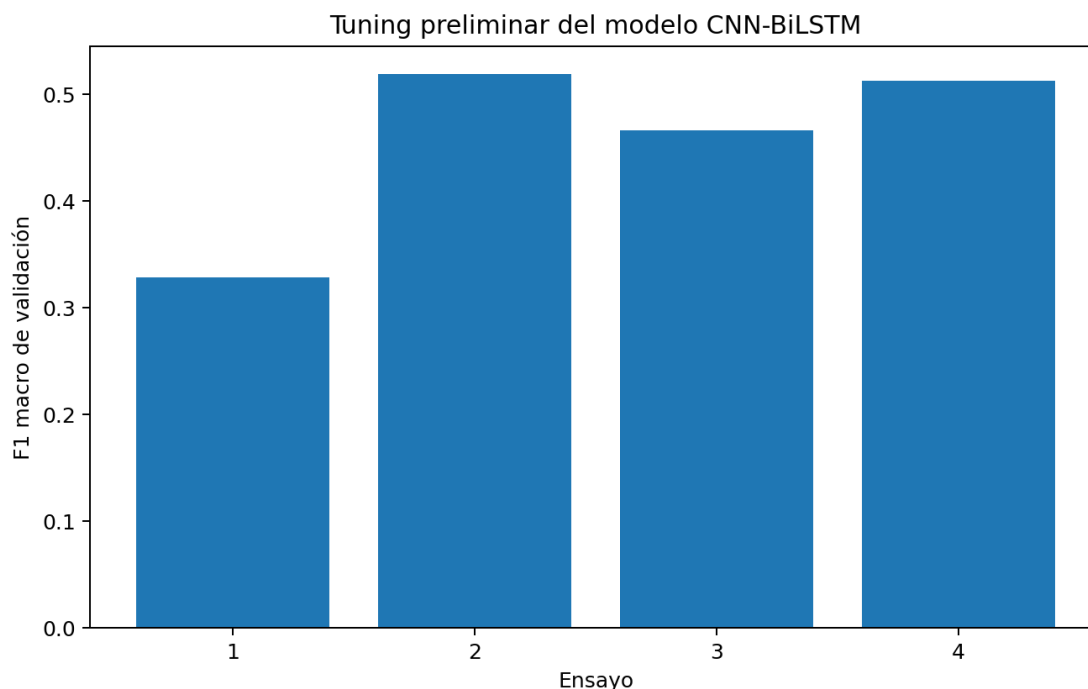


Figura 10. F1 macro de los ensayos de tuning.

La mejor combinación fue embedding 32, 32 unidades ocultas, dropout 0,20 y learning rate 0,001, con F1 macro de validación 0,5189. La configuración con mayor accuracy no coincidió con la de mayor F1 macro, por lo que el criterio de selección se mantuvo alineado con el objetivo de tratar todas las clases de forma equilibrada.

4.6 Pruebas estadísticas

Prueba	Datos	Estadístico	p-valor	Conclusión $\alpha=0,05$
Friedman	F1 macro de 5 folds / 5 modelos	17,2800	0,001705	Sí, diferencia global
Wilcoxon + Holm	10 comparaciones pareadas	p ajustada mínima 0,625	$\geq 0,625$	No

Tabla 8. Resultados de las pruebas estadísticas.

La prueba de Friedman permitió rechazar H_0 a nivel global. Sin embargo, ninguna comparación pareada conservó significancia después de la corrección de Holm. Esto no contradice el resultado global: con solo cinco observaciones por modelo, las pruebas pareadas tienen baja resolución y la corrección múltiple es conservadora. La conclusión correcta es que se detectan diferencias conjuntas, pero no hay evidencia suficiente para afirmar qué par específico difiere con significancia ajustada.

4.7 Persistencia, API y reportes

El artefacto seleccionado fue MLP-TFIDF y se guardó como models/best_model.h5 junto con best_model_metadata.json, labels.json y tfidf_vectorizer.joblib. FastAPI carga el archivo H5 directamente, con un respaldo .pt, y devuelve etiqueta, nombre de clase, confianza, probabilidades y nombre del artefacto. Las pruebas automáticas verificaron salud, disponibilidad y predicción sin reentrenamiento.

Resultado funcional: El endpoint /health respondió status=ok y /predict consumió best_model.h5. Los reportes se generaron en Word, PDF y Excel; el Excel contiene dashboard, EDA, métricas, validación cruzada, tuning, pruebas y reportes por clase.

5. Discusión

El resultado principal favoreció a MLP-TFIDF en el conjunto de prueba. En intervenciones breves, las distribuciones léxicas resultaron muy informativas y permitieron competir con modelos secuenciales más complejos. Además, el modelo fue más rápido y pequeño. Esto es valioso para despliegues con recursos limitados y para una API que debe responder sin una infraestructura de cómputo especializada.

CNN-1D fue el modelo más estable en validación cruzada y logró la mayor accuracy en prueba. Por tanto, aunque MLP-TFIDF fue seleccionado por F1 macro oficial, CNN-1D es una alternativa razonable cuando se prioriza estabilidad entre reuniones. Los modelos híbridos no superaron a los clásicos bajo la configuración de dos épocas. Su mayor complejidad incrementó el tiempo, sin garantizar mejor generalización.

El desempeño de las clases de retroalimentación y seguimiento sugiere que una intervención aislada no siempre contiene suficiente información. Como MRDA es conversacional, incorporar contexto de turnos anteriores, información del hablante o modelos preentrenados podría elevar el rendimiento. No obstante, el objetivo docente se centró en comparar modelos clásicos e híbridos entrenados por el equipo, requisito que se cumplió sin utilizar resultados simulados.

La integración de FastAPI y Streamlit convierte el experimento en una aplicación funcional. La separación entre frontend, backend y entrenamiento permite que el modelo se entrene una vez, se guarde y posteriormente sea consumido por el sistema. Los reportes y figuras conservan trazabilidad hacia archivos CSV generados por el experimento.

6. Conclusiones

- Se procesaron 109 228 intervenciones reales de un dataset público de reuniones, sin duplicados de identificador ni textos vacíos en las variables analizadas.
- Se implementaron tres modelos clásicos y dos híbridos. MLP-TFIDF obtuvo el mejor F1 macro en prueba (0,6219), mientras CNN-1D presentó la mejor estabilidad en validación cruzada.
- La accuracy no fue suficiente para seleccionar el mejor modelo: CNN-1D tuvo mayor accuracy, pero MLP-TFIDF mostró mejor equilibrio macro entre clases.
- La prueba de Friedman confirmó diferencias globales; las comparaciones Wilcoxon-Holm no identificaron pares significativos con cinco folds.
- El mejor modelo se persistió en .h5 y es consumido directamente por FastAPI sin reentrenamiento. El dashboard Streamlit muestra predicción, EDA, modelos, validación y reportes después de autenticación.
- La solución produce PDF, Word y Excel con tablas, figuras e interpretación, y queda preparada para control de versiones en GitHub, documentación en Jira y despliegue en Render/Streamlit o mediante una landing en Vercel.

7. Limitaciones y trabajo futuro

- El corpus está en inglés, mientras que la interfaz de gestión está en español. No debe asumirse el mismo desempeño con transcripciones en español. Para el uso práctico, la aplicación desplegada incorpora una capa de traducción automática español→inglés previa a la predicción, declarada en la propia interfaz; como trabajo futuro se propone el reentrenamiento con un corpus de actos de diálogo en español (por ejemplo, DIHANA).
- La comparación principal entrenó con una muestra estratificada de 30 000 registros por restricciones de cómputo, aunque el test oficial completo fue utilizado.
- Se ejecutaron dos épocas, por lo que los resultados son una línea base y no un máximo optimizado.
- Cinco folds cumplen el requisito, pero ofrecen poca potencia para comparaciones pareadas después de Holm.
- El sistema fue desplegado públicamente el 12 de julio de 2026 con las cuentas del equipo: API FastAPI en Render (<https://reuniones-ia-api.onrender.com>), frontend Streamlit en Render (<https://reuniones-ia-frontend.onrender.com>), landing en Vercel (<https://reuniones-ia-articulo.vercel.app>), código con integración continua y release v1.0.0 en GitHub (<https://github.com/lizbethGuzman16/reuniones-ia-articulo>) y gestión del proyecto en Jira (proyecto RIA). Los servicios usan planes gratuitos, por lo que la primera petición tras un periodo de inactividad puede tardar hasta dos minutos.

- Como trabajo futuro se propone entrenar con todo el split, incorporar contexto conversacional, evaluar modelos preentrenados y construir o validar un corpus equivalente en español.

Referencias

- Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural machine translation by jointly learning to align and translate. *International Conference on Learning Representations*.
- Chapuis, E., Colombo, P., Manica, M., Labeau, M., & Clavel, C. (2020). Hierarchical pre-training for sequence labelling in spoken dialog. *Findings of EMNLP 2020*, 2636–2648. <https://doi.org/10.18653/v1/2020.findings-emnlp.239>
- Demšar, J. (2006). Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7, 1–30.
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
- Holm, S. (1979). A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2), 65–70.
- Kim, Y. (2014). Convolutional neural networks for sentence classification. *Proceedings of EMNLP 2014*, 1746–1751. <https://doi.org/10.3115/v1/D14-1181>
- Paszke, A., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32.
- Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Schuster, M., & Paliwal, K. K. (1997). Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing*, 45(11), 2673–2681. <https://doi.org/10.1109/78.650093>
- Shriberg, E., Dhillon, R., Bhagat, S., Ang, J., & Carvey, H. (2004). The ICSI Meeting Recorder Dialog Act (MRDA) Corpus. *Proceedings of the 5th SIGdial Workshop*, 97–100.
- SILICONE Benchmark. (2026). MRDA public data and benchmark repository. <https://github.com/eusip/SILICONE-benchmark>

Anexo A. Matrices de confusión de los cinco modelos

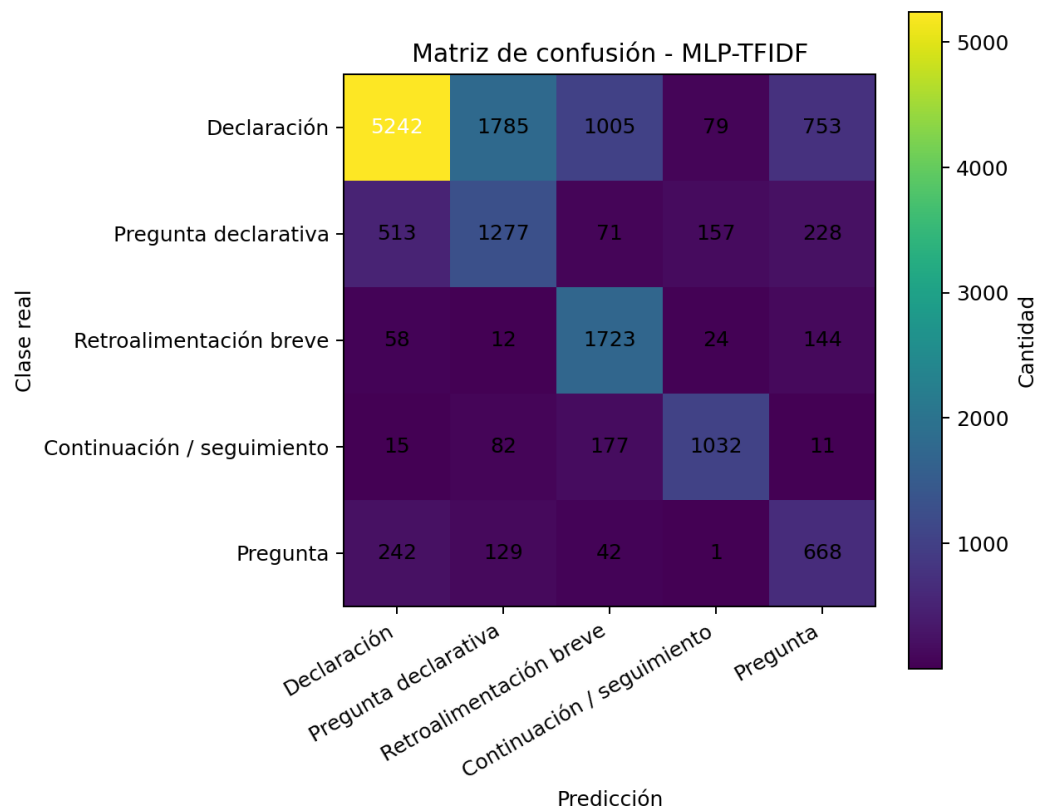


Figura A1. Matriz de confusión: MLP-TFIDF.

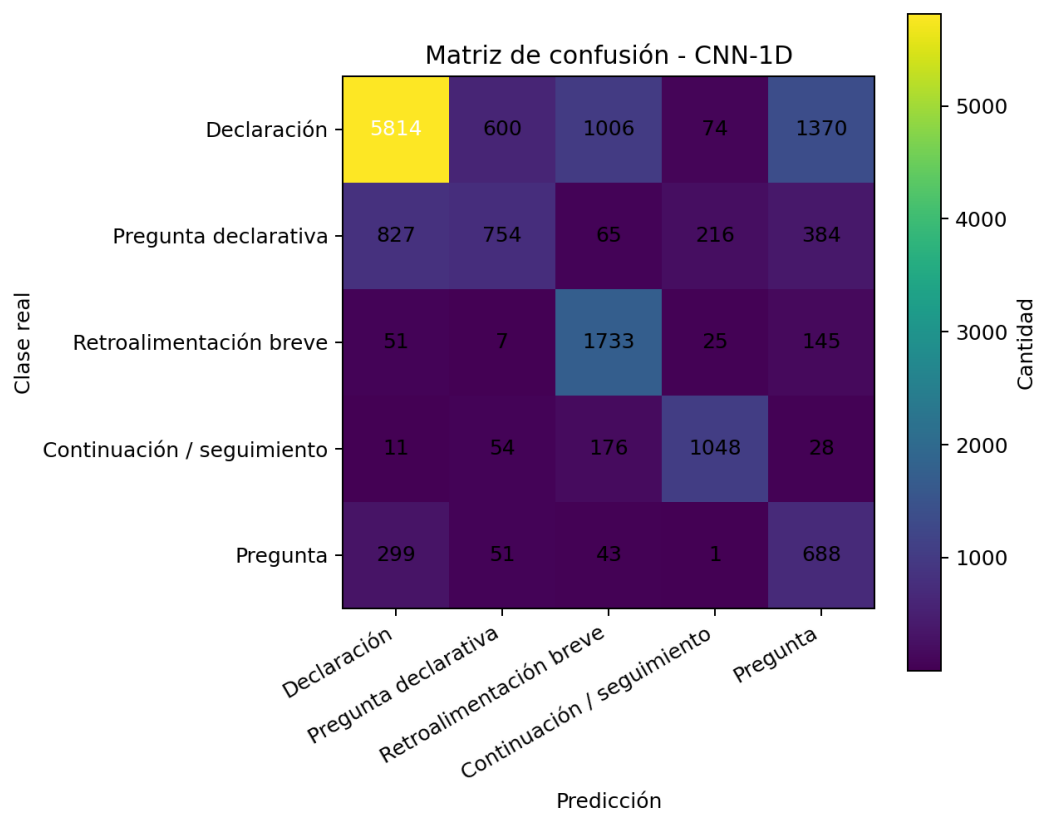


Figura A2. Matriz de confusión: CNN-1D.

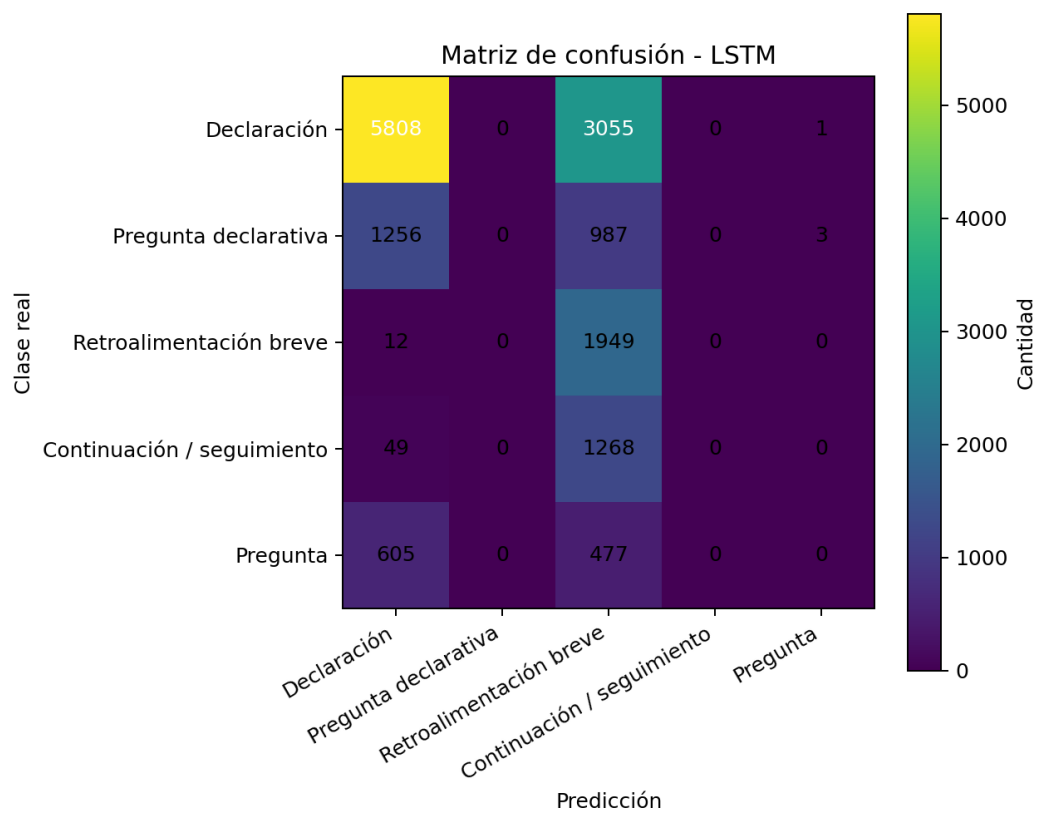


Figura A3. Matriz de confusión: LSTM.

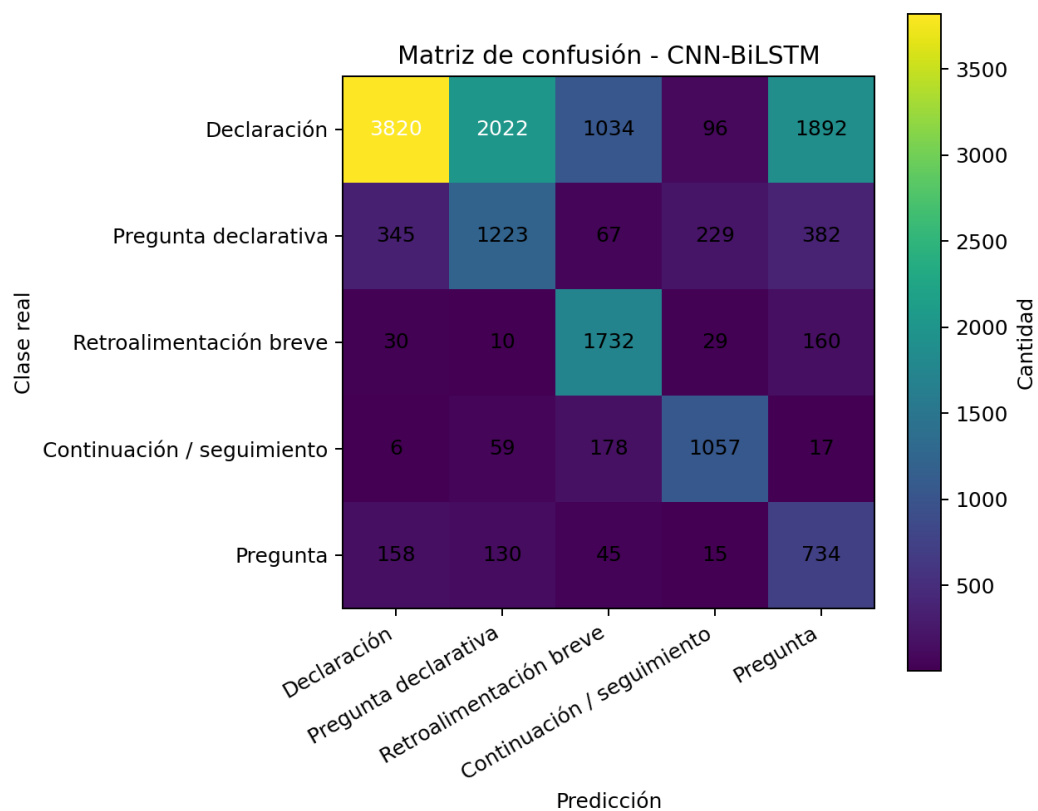


Figura A4. Matriz de confusión: CNN-BiLSTM.

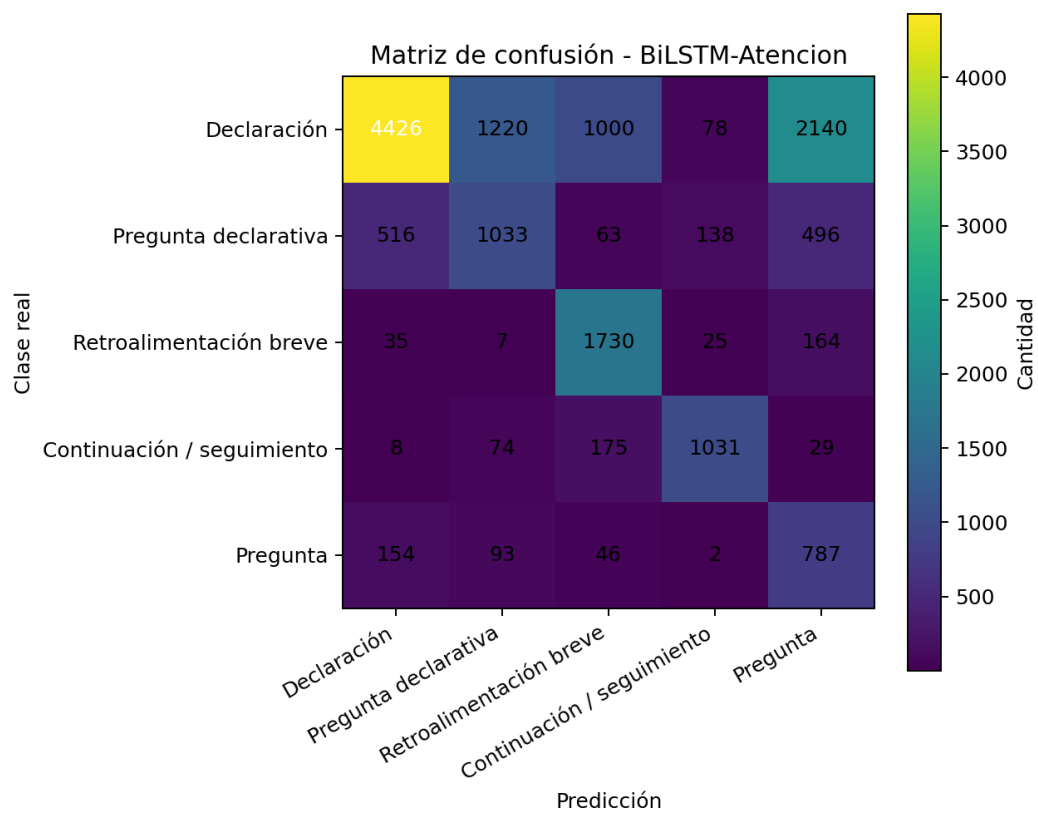


Figura A5. Matriz de confusión: BiLSTM con atención.

Anexo B. Curvas ROC de los cinco modelos

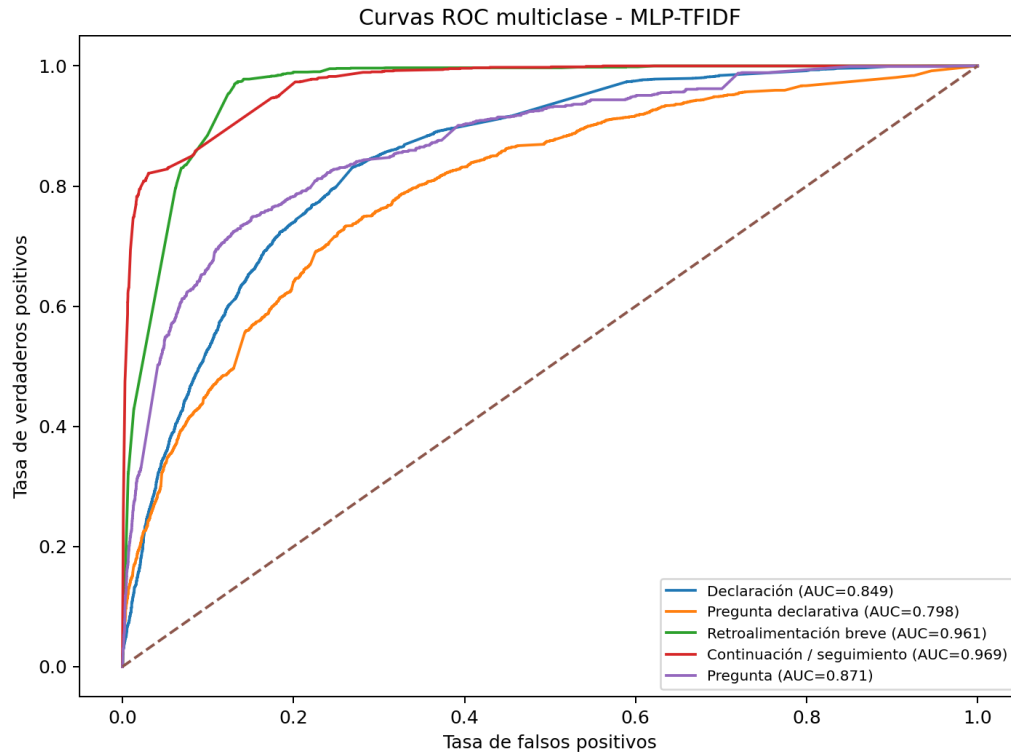


Figura B1. Curvas ROC: MLP-TFIDF.

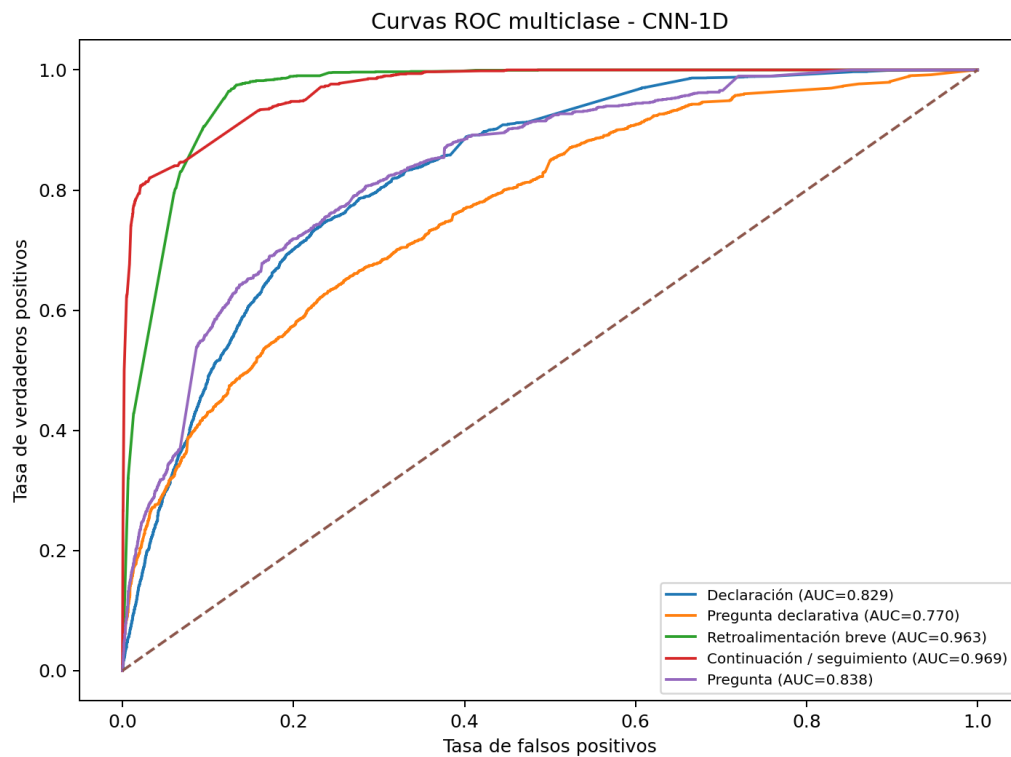


Figura B2. Curvas ROC: CNN-1D.

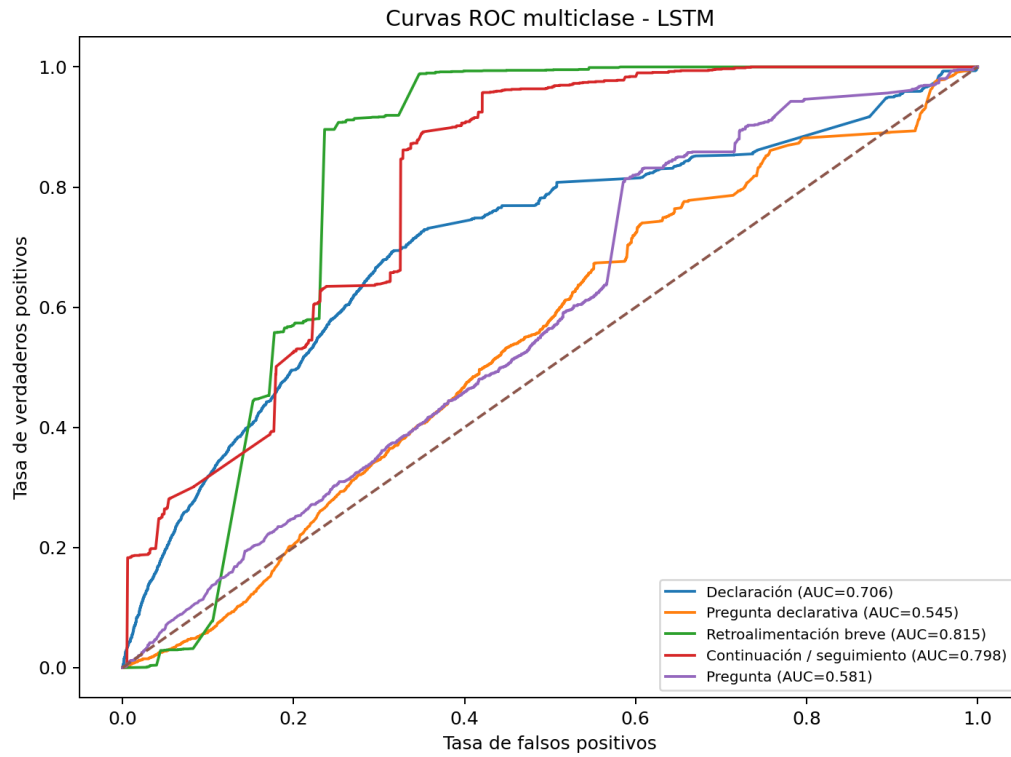


Figura B3. Curvas ROC: LSTM.

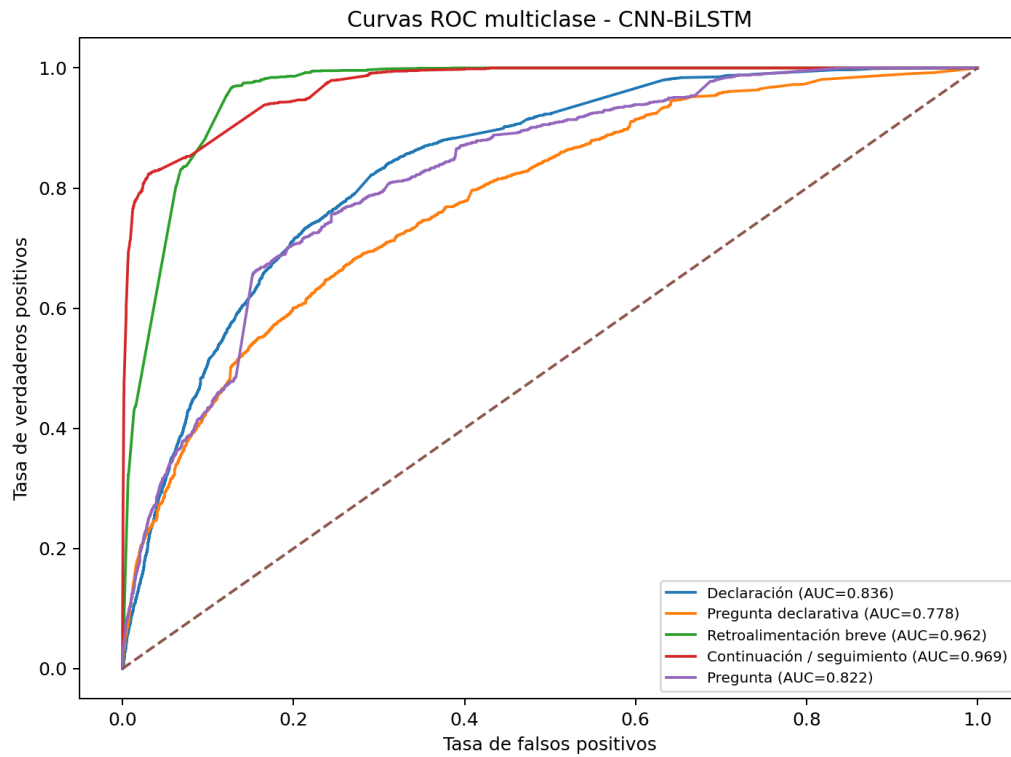


Figura B4. Curvas ROC: CNN-BiLSTM.

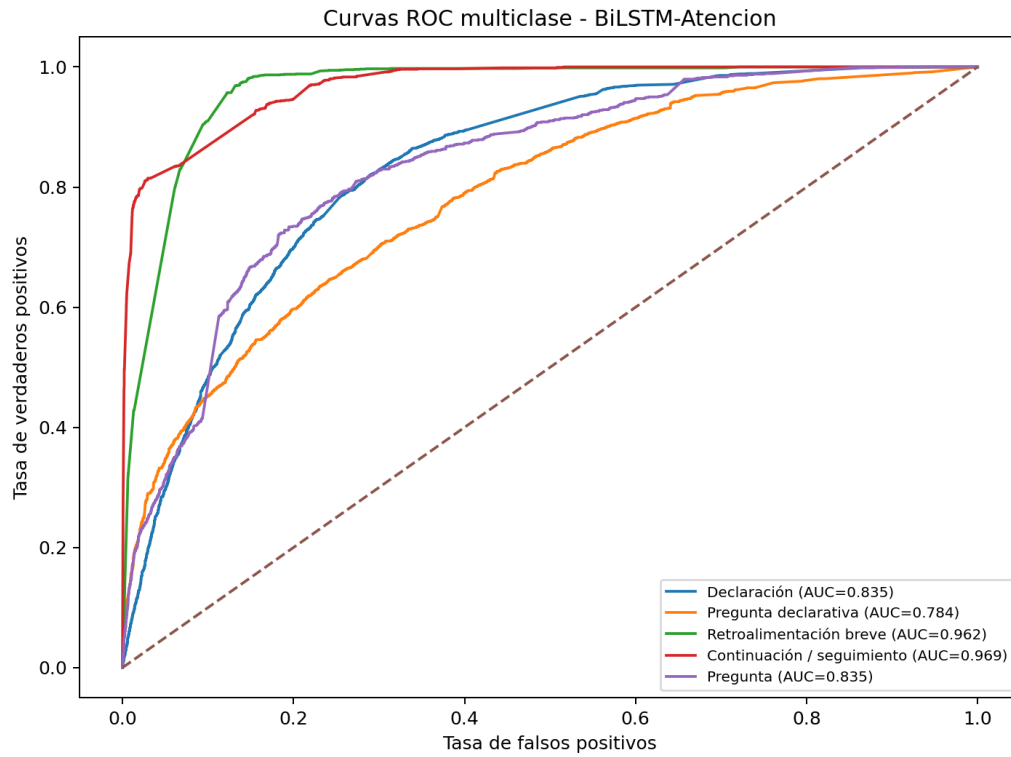


Figura B5. Curvas ROC: BiLSTM con atención.

Anexo C. Historiales de pérdida

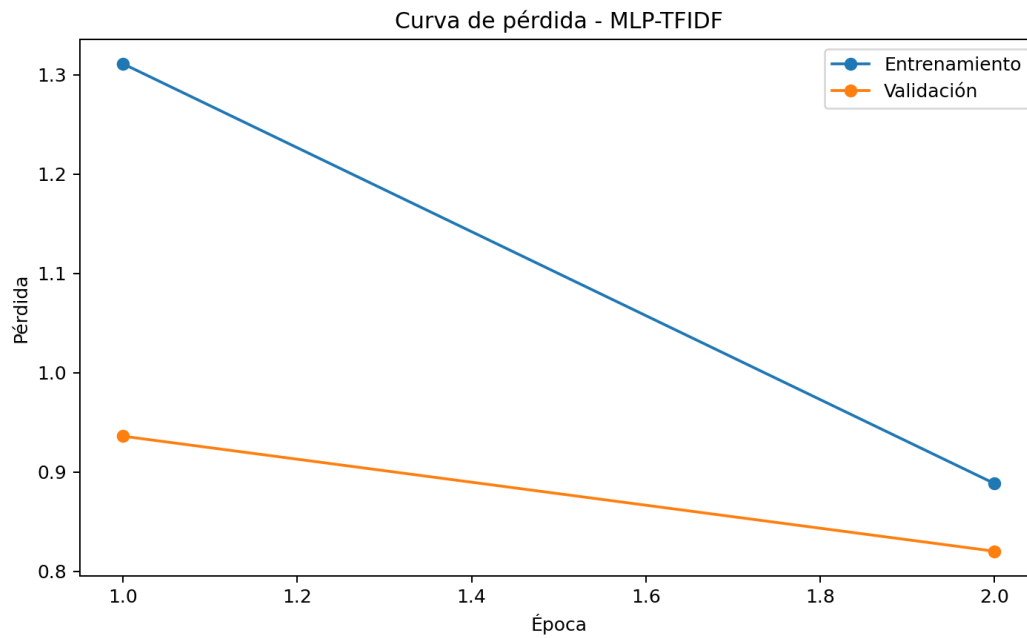


Figura C1. Pérdida por época: MLP-TFIDF.

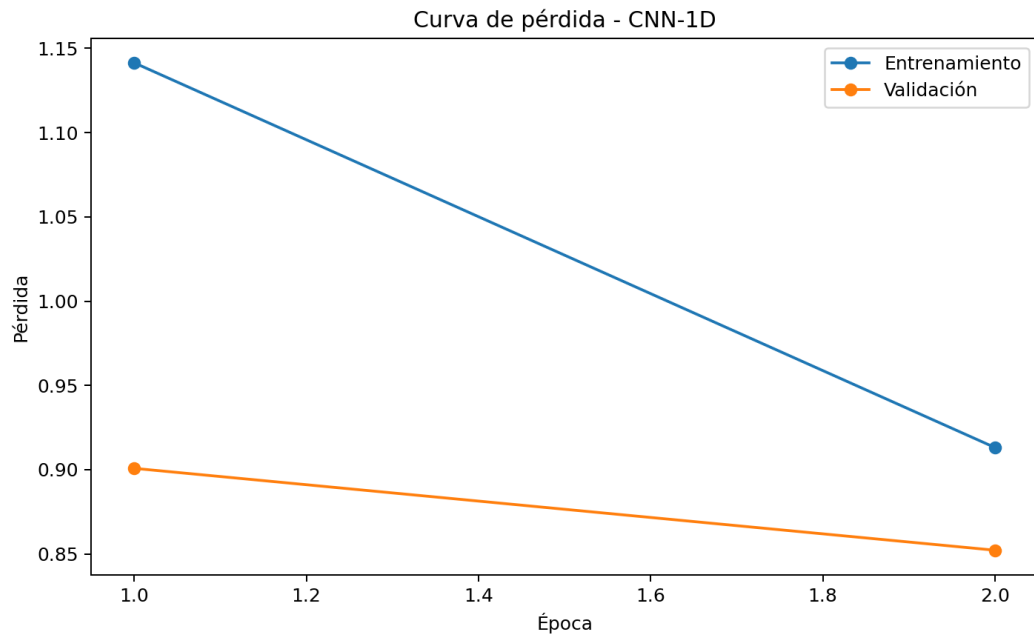


Figura C2. Pérdida por época: CNN-1D.

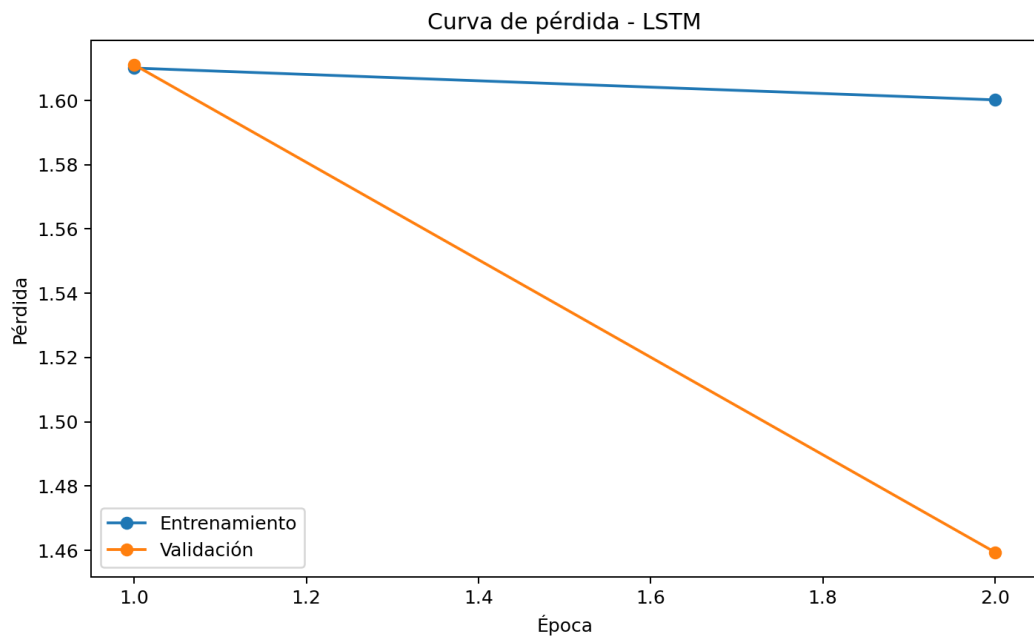


Figura C3. Pérdida por época: LSTM.

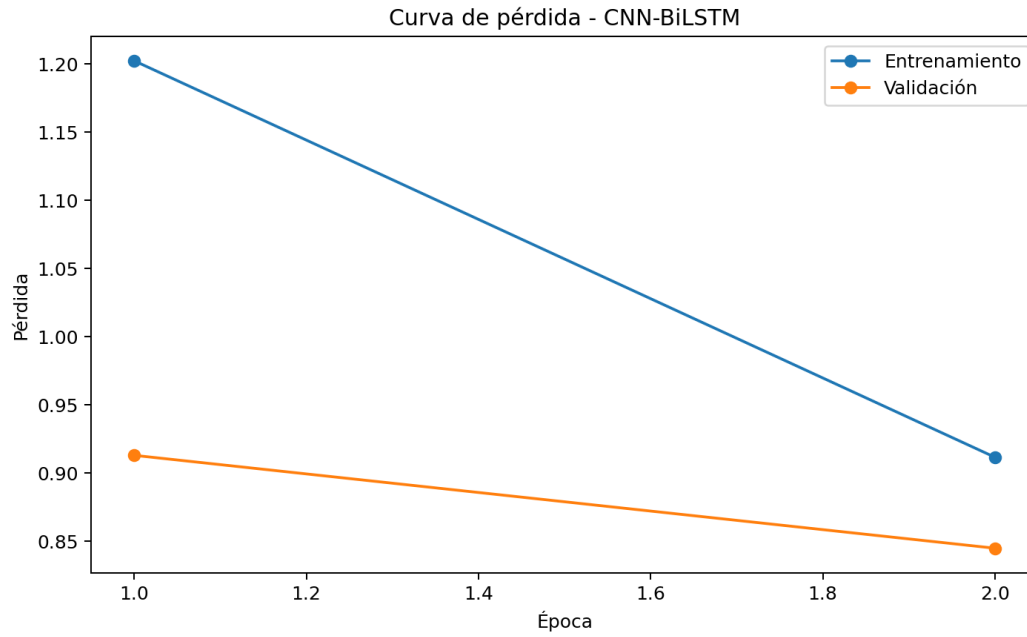


Figura C4. Pérdida por época: CNN-BiLSTM.

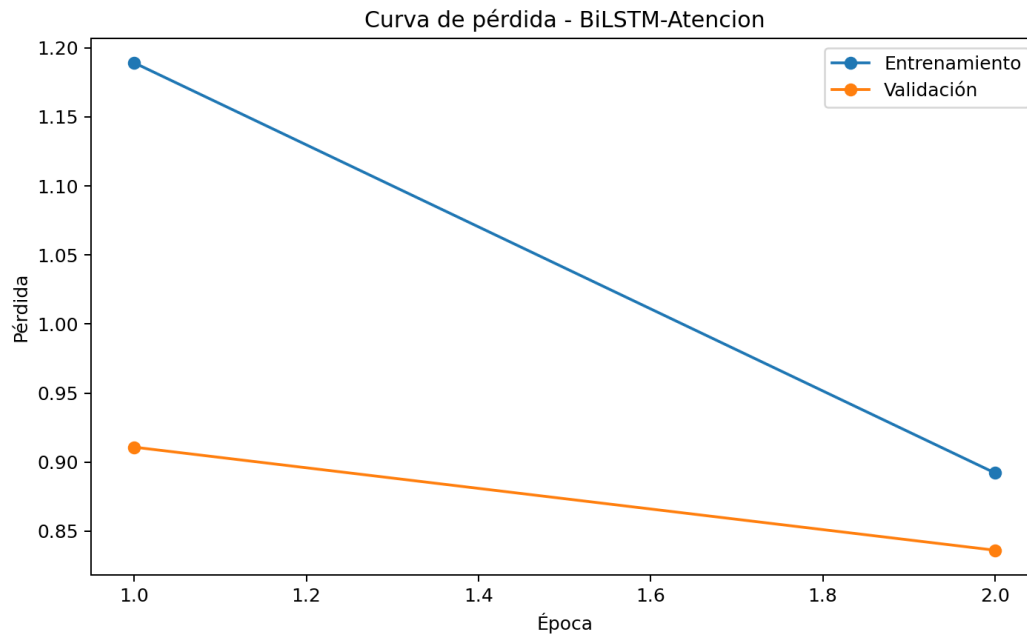


Figura C5. Pérdida por época: BiLSTM con atención.

Anexo D. Matriz de cumplimiento del requerimiento docente

Requisito	Estado	Evidencia
Dataset público	Sí	MRDA/SILICONE, manifiesto con URL, filas y SHA-256
EDA y limpieza	Sí	Nulos, duplicados, clases, estadísticos, longitudes, palabras y heatmap
3 modelos clásicos	Sí	MLP-TFIDF, CNN-1D, LSTM
2 modelos híbridos	Sí	CNN-BiLSTM, BiLSTM-Atención
Tablas comparativas	Sí	Métricas, tiempo, inferencia, tamaño y parámetros
Matriz de confusión	Sí	Cinco matrices

Requisito	Estado	Evidencia
Mapa de calor	Sí	Clases por partición
Curva ROC	Sí	Cinco gráficos multiclase
Modelo .h5	Sí	best_model.h5 consumido por FastAPI
Cross validation	Sí	5 folds configurables y agrupados por reunión
Tuning	Sí	4 ensayos CNN-BiLSTM
Pruebas robustas	Sí	Friedman + Wilcoxon-Holm
PDF, Word y Excel	Sí	Generados desde resultados reales
Dashboard con credenciales	Sí	Módulo IA posterior a autenticación; demo local identificada
Streamlit + FastAPI	Sí	Frontend y backend separados
Render/Vercel	Preparado	Archivos listos; publicación requiere cuentas del equipo
GitHub	Preparado	.gitignore, CI y documentación
Jira	Preparado	CSV importable de backlog

Tabla D1. Cumplimiento técnico y científico.

Anexo E. Reproducibilidad

Comandos principales desde la raíz del proyecto:

```
python -m ml.download_mrda

python -m ml.eda

python -m ml.train_all

python -m ml.cross_validation

python -m ml.tuning

python -m ml.statistical_tests

uvicorn backend.main:app --host 0.0.0.0 --port 8000

streamlit run frontend/app.py

pytest -q
```

Los valores configurables se encuentran en ml/config.py. La semilla de la entrega es 42. Los archivos CSV de resultados son la fuente de las tablas del Word, PDF y Excel.

Anexo F. Evidencias reales de ejecución

Las siguientes capturas fueron obtenidas ejecutando el frontend Streamlit y el backend FastAPI. La franja amarilla identifica el modo de demostración local para la autenticación y los módulos originales; las métricas, el modelo H5 y las predicciones proceden de los artefactos reales generados con MRDA.

Adicionalmente, el sistema quedó desplegado públicamente y puede verificarse en línea: API <https://reuniones-ia-api.onrender.com> (salud en /health, documentación en /docs), aplicación <https://reuniones-ia-frontend.onrender.com>, landing <https://reuniones-ia-articulo.vercel.app> y repositorio <https://github.com/lizbethGuzman16/reuniones-ia-articulo> (release v1.0.0 con los reportes adjuntos).

Pretty-print ☐

```
{"status": "ok", "service": "reuniones-ia-api"}
```

Figura F1. Respuesta real del endpoint de salud de FastAPI.

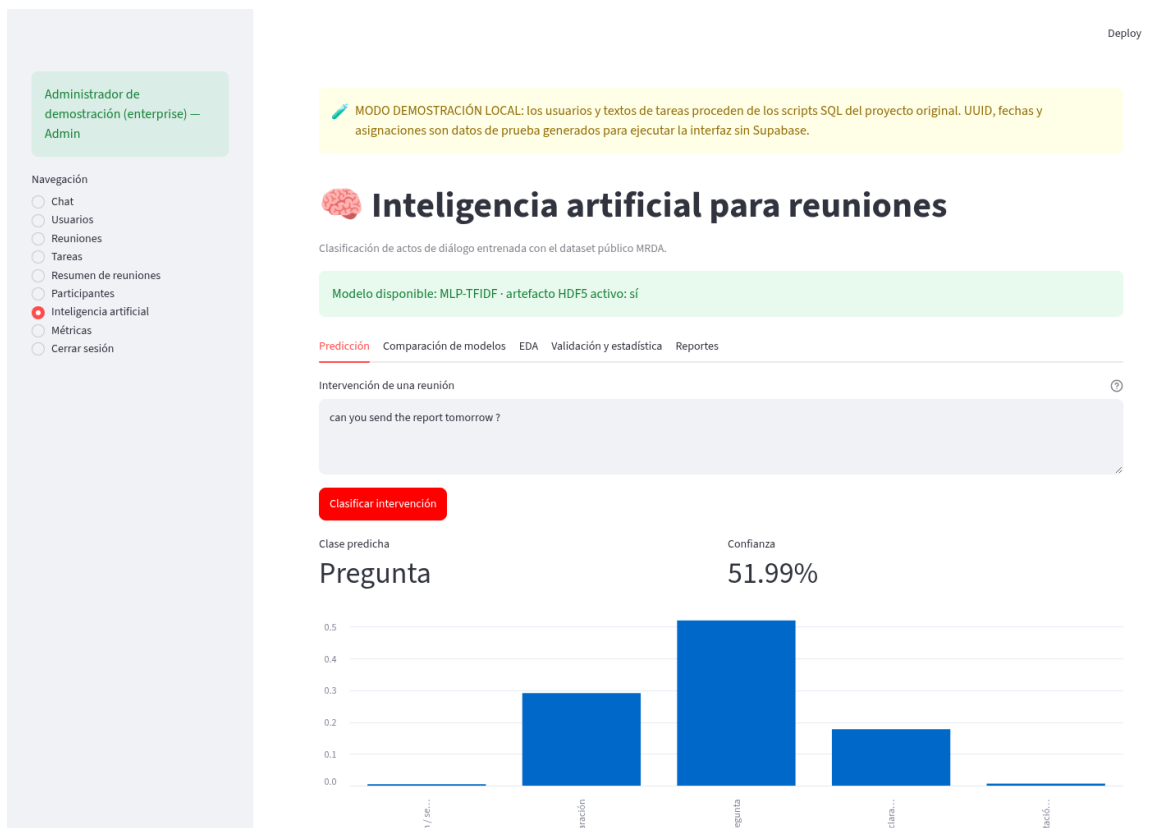
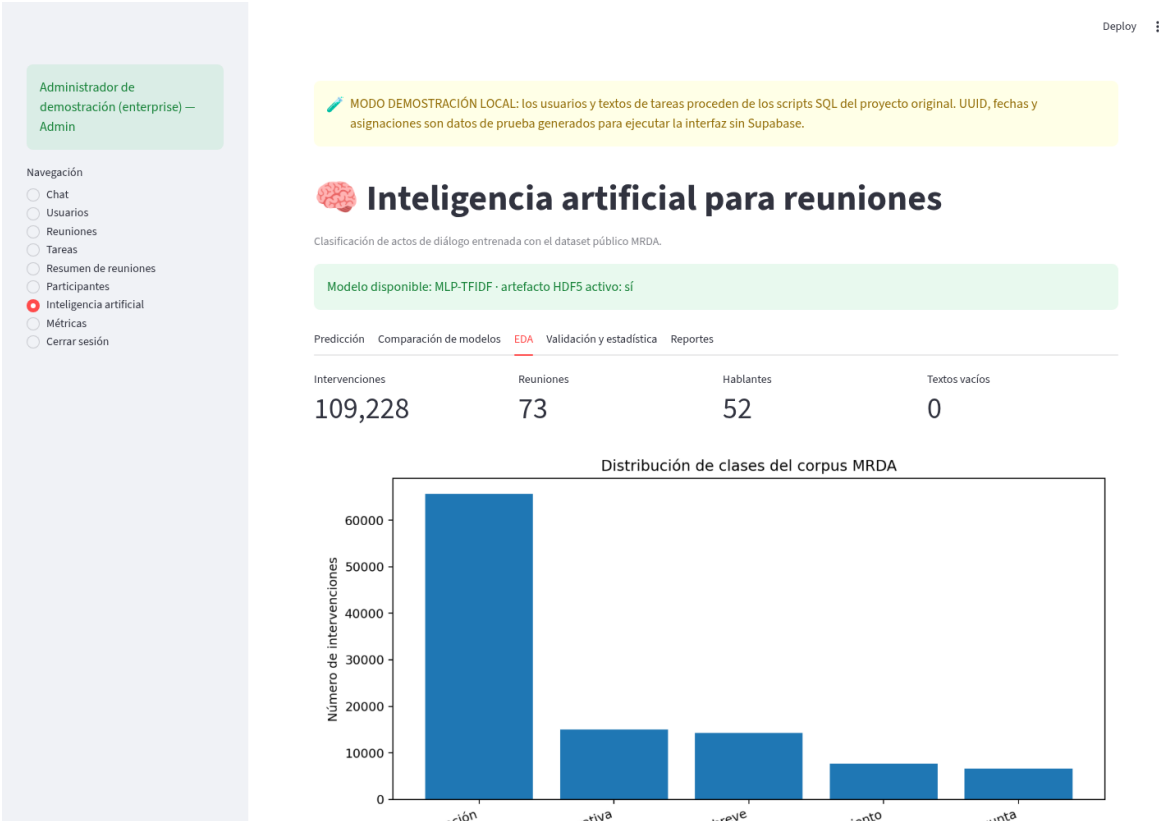
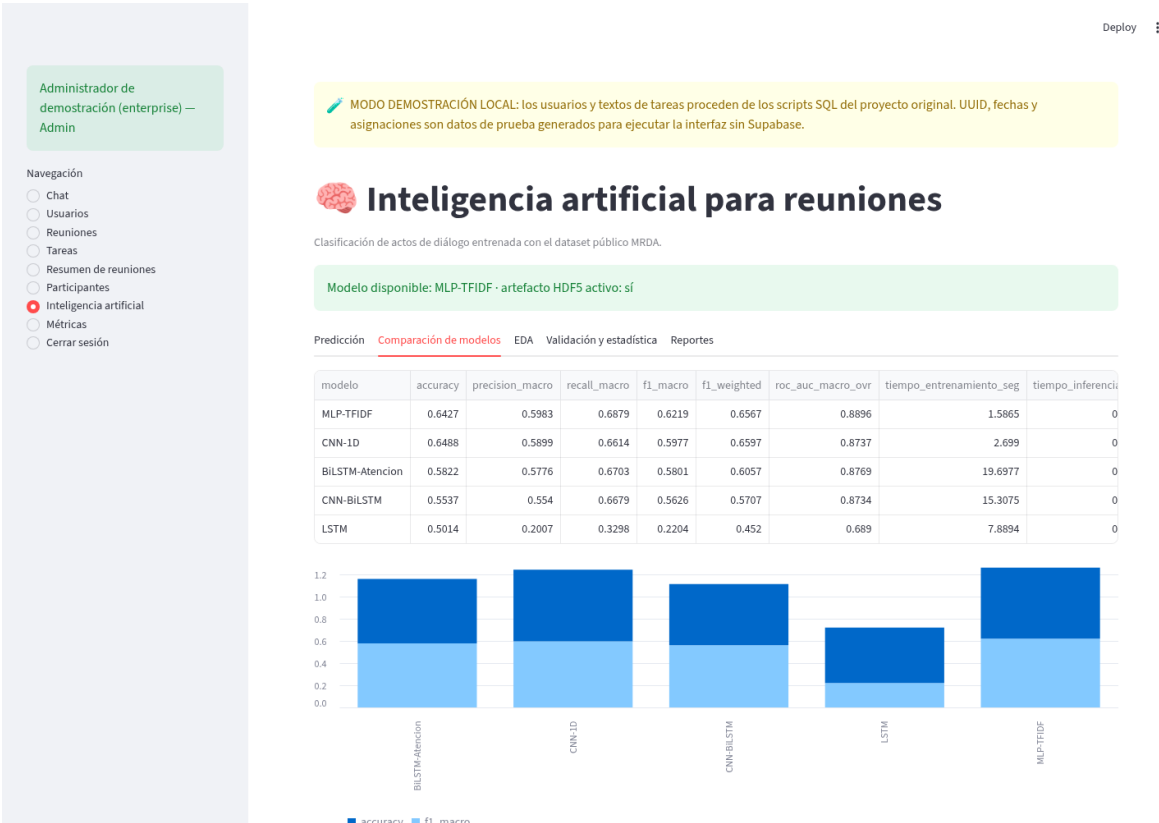


Figura F2. Predicción real consumiendo best_model.h5.



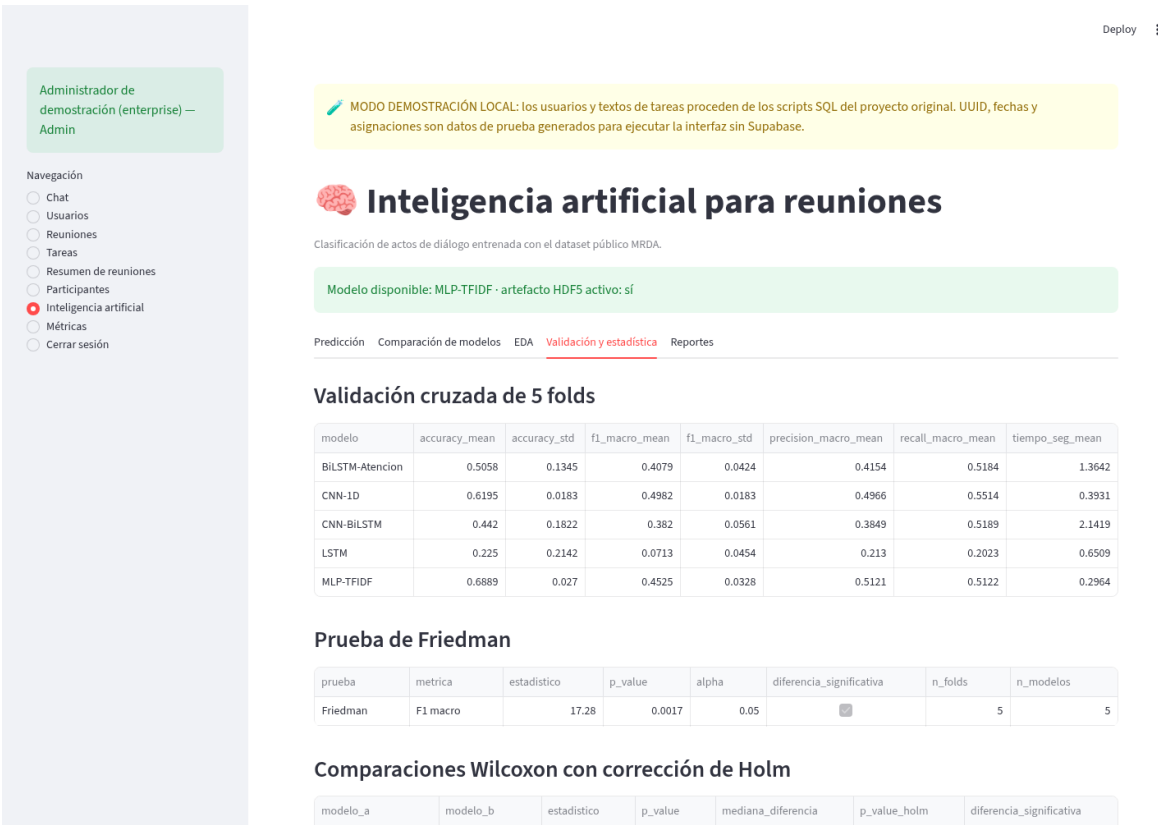


Figura F5. Validación cruzada y pruebas estadísticas en pantalla.

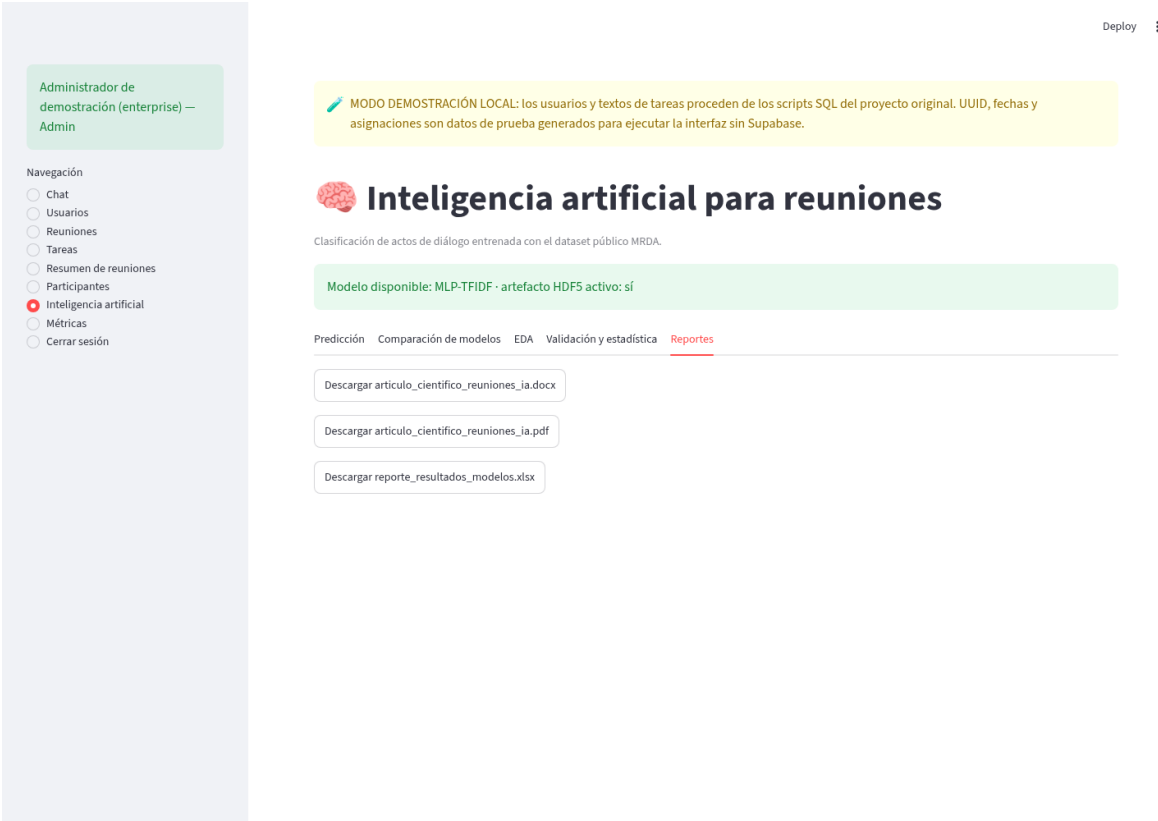


Figura F6. Descarga de Word, PDF y Excel desde Streamlit.